

Архитектура квеста

«Выход из квартиры»

1 Общая структура

Проект построен на базе интерфейсов и абстрактных классов, что обеспечивает гибкость и расширяемость. Основные компоненты:

- **Game** – главный класс, управляющий игровым циклом, локациями, состоянием и командами.
- **GameState** – хранилище всего состояния игры (здоровье, инвентарь, флаги, счётчик ходов, журнал).
- **Location** – локация с названием, описанием, объектами, событиями и выходами.
- **IInteractable** – объекты, с которыми можно взаимодействовать.
- **ICondition** – условия для проверки состояния игры.
- **IEffect** – эффекты, изменяющие состояние.
- **GameEventBase** – события, срабатывающие автоматически.
- **ICommand** – команды пользователя.
- **Quest** – система заданий.

2 Интерфейсы

ICommand

```
void Execute(Game game, string[] args)
```

Выполнение команды пользователя.

IInteractable

```
string Id { get; }
```

```
string Name { get; }
```

```
string Description { get; }
```

```
void Interact(GameState state)
```

Взаимодействие с объектом мира.

ICondition

```
bool IsTrue(GameState state)
```

Проверка условия.

IEffect

`void Apply(GameState state)`

Применение эффекта к состоянию игры.

3 Абстрактные классы

CommandBase

реализует `ICommand`

База для всех команд. Может содержать общие поля (например, название команды).

InteractableBase

реализует `IInteractable`

База для объектов взаимодействия. Содержит `Id`, `Name`, `Description`. Абстрактный метод `Interact`.

ConditionBase

реализует `ICondition`

База для условий. Абстрактный метод `IsTrue`.

EffectBase

реализует `IEffect`

База для эффектов. Абстрактный метод `Apply`.

GameEventBase

Содержит:

`ICondition Condition` – условие срабатывания.

`List<IEffect> Effects` – список эффектов.

`bool IsOneTime` – признак одноразового события.

Абстрактный метод `void CheckAndApply(GameState state)` – проверяет условие и применяет эффекты.

4 Команды (наследуют CommandBase)

- **LookCommand** – описание текущей локации и список объектов.
- **GoCommand** – перемещение в соседнюю локацию.
- **InteractCommand** – взаимодействие с объектом по `Id`.
- **InventoryCommand** – содержимое инвентаря.
- **HelpCommand** – список доступных команд.
- **StatusCommand** – здоровье, флаги, счётчик ходов.

5 Условия (наследуют ConditionBase)

- **HasItemCondition** (`string itemId`) – наличие предмета.
- **FlagCondition** (`string flagName, bool expectedValue`) – значение флага.
- **HealthCondition** (`int threshold, Comparison comparison`) – сравнение здоровья.

6 Эффекты (наследуют EffectBase)

- **AddItemEffect** (string itemId) – добавить предмет.
- **RemoveItemEffect** (string itemId) – удалить предмет.
- **SetFlagEffect** (string flagName, bool value) – установить флаг.
- **DamageEffect** (int amount) – нанести урон.
- **HealEffect** (int amount) – восстановить здоровье.
- **LogEffect** (string message) – запись в журнал.
- **AddExitEffect** (string direction, string targetLocationId) – добавить выход.
- **RemoveExitEffect** (string direction) – удалить выход.
- **ChangeLocationEffect** (string locationId) – переместить игрока.
- **CompleteQuestEffect** (string questId, bool success) – завершить квест.
- **StartQuestEffect** (string questId) – начать квест.

7 Объекты взаимодействия (наследуют InteractableBase)

- **Chest** – сундук.
Поля: ICondition? OpenCondition, List<IEffect> OpenEffects, bool IsOpen.
При взаимодействии, если закрыт и условие выполнено, применяет эффекты и становится открытым.
- **Door** – дверь.
Поля: ICondition? CanPassCondition, string TargetLocationId, bool IsLocked.
При взаимодействии, если условие пропускает, перемещает игрока в целевую локацию (через ChangeLocationEffect).
- **NPC** – непись.
Поля: Dictionary<ICondition, List<IEffect> DialogOptions.
При взаимодействии проверяет условия и выполняет соответствующие эффекты (диалог, выдача квеста).
- **Trap** – ловушка.
Поля: ICondition? TriggerCondition, List<IEffect> TriggerEffects, bool IsOneTime.
При взаимодействии, если условие срабатывает, применяет эффекты и, если одноразовая, деактивируется.
- **SimpleObject** – простой объект (кровать, картина и т.п.).
Поля: List<IEffect> OnInteractEffects.
Всегда применяет эффекты при взаимодействии.

8 События (наследуют `GameEventBase`)

- `OnEnterLocationEvent` – проверяется при входе в локацию.
- `OnTurnEvent` – проверяется каждый ход.
- `OneTimeEvent` – проверяется постоянно, но срабатывает только один раз.

9 Квесты

Класс `Quest` содержит:

- `string Id`
- `string Name`
- `string Description`
- `QuestState State` (`NotStarted`, `InProgress`, `Completed`)
- `ICondition CompletionCondition` – условие завершения.
- Метод `void Update(GameState state)` – проверяет условие и обновляет состояние.

10 Игровое состояние

Класс `GameState` содержит:

- `int Health`
- `List<string> Inventory`
- `Dictionary<string, bool> Flags`
- `int TurnCount`
- `List<string> EventLog`
- Методы: `AddItem`, `RemoveItem`, `HasItem`, `SetFlag`, `GetFlag`, `Damage`, `Heal`, `AddLog`, `IncrementTurn`.

11 Локация

Класс `Location` содержит:

- `string Id`
- `string Name`
- `string Description`
- `List<IInteractable> Interactables`
- `List<GameEventBase> Events`

- Dictionary<string, string> Exits (ключ – направление, значение – Id локации)
- Методы: AddExit, RemoveExit, GetExit.

12 Главный класс игры

Класс **Game** содержит:

- GameState State
- Dictionary<string, Location> Locations
- string CurrentLocationId
- List<Quest> Quests
- bool IsRunning
- Методы: Run() – игровой цикл; ProcessCommand(string input) – разбор и выполнение команд; CheckEvents(EventType type) – проверка событий; UpdateQuests() – обновление квестов.

13 Схема локаций

Спальня $\longleftrightarrow$ Гостиная $\longleftrightarrow$ Тёмный коридор $\longleftrightarrow$ Ванная
 Гостиная $\longrightarrow$ Кухня (скрытая) $\longrightarrow$ Ванная
 Ванная $\longrightarrow$ Прихожая $\longrightarrow$ Выход

14 Игровые механики (реализованные через условия и эффекты)

1. **Тёмный коридор** – событие OnEnterLocationEvent с условием NotCondition(HasItemCondition) и эффектом DamageEffect(10).
2. **Одноразовая ловушка** – объект Trap в ванной (душевая шторка) с эффектом DamageEffect(100) и флагом IsOneTime = true.
3. **Дверь с ключом** – объект Door в прихожей с условием HasItemCondition("Key") и эффектом ChangeLocationEffect("Outside").
4. **NPC с диалогом** – объект NPC на кухне: диалоговые опции проверяют наличие "OldPhoto" и "Cup", выдают квест GhostHelp и ключ.
5. **Несколько вариантов завершения** – выход через дверь (с ключом) или смерть от ловушки / истощения здоровья.

15 Диаграмма классов (текстовое описание связей)

- `Game` содержит `GameState`, `Location[]`, `Quest[]`, `CurrentLocationId`.
- `Location` содержит списки `IInteractable`, `GameEventBase`, словарь `Exits`.
- `GameEventBase` реализует `ICondition` и `IEffect`.
- `InteractableBase` реализует `IInteractable`. Конкретные объекты (`Chest`, `Door` и т.д.) наследуют `InteractableBase` и могут содержать `ICondition` и `IEffect`.
- `ConditionBase` реализует `ICondition`.
- `EffectBase` реализует `IEffect`.
- `CommandBase` реализует `ICommand`. Команды используют `Game` для доступа к состоянию и локациям.
- `Quest` содержит `ICondition` для проверки завершения.